Modularisierung

Joern Ploennigs · AI4SC



> Write programs that do one thing and do it well.
>
> — Doug McIlroy

Projektstruktur als Architekturentscheidung

Größere Programme enthalten oft dutzende Klassen mit hunderten Funktionen und vielen tausend Zeilen Code. Hier wird es sehr schnell schwer einen Überblick zu behalten, wenn alle Klassen in einer Datei definiert sind. Insbesondere wenn verschiedene Programmierer an verschiedenen Stellen im Programm arbeiten, kommt es dann sehr schnell zu Versionskonflikten, wenn Leute an ähnlichen Dateien arbeiten.

Um dies organisiert und übersichtlich zu halten, wird Code in mehrere Dateien mit der Erweiterung `.py` aufgespalten. Dabei ist es üblich jeweils eine Datei

- pro Klasse, wenn Klassen definiert werden
- pro Thema, wenn Hilfsfunktionen definiert werden (z.B. Mathematik-Funktionen, ...)
- pro Aufgabenbereich, wenn (z.B. Laden von Daten getrennt von deren Verarbeitung. So kann man z.B. später andere Verarbeitungsschritte definieren und das Laden wiederverwenden)

Modulgrenzen sind Architekturentscheidungen: wo ein Modul endet und das nächste beginnt, legt fest, wie KI-generierter Code in das Projekt eingebettet wird.

Speichern wir jede Klasse der Geometrieelemente aus dem Teil der Klassendefinition aus der letzten Vorlesung so hätten wir dann z.B. eine Projektstruktur entsprechend:

- 📁 geometry
  - 📄 ImmutablePoint.py
  - 📄 Point.py

- Line.py
- Pentagon.py
- Polygon.py
- Tetragon.py
- Triangle.py

Dabei enthält jede Datei nur den Code der gleichnamigen Klasse, auch wenn dies nur wenige Zeilen sind, wie im Fall der Klassen `Triangle`, `Tetragon` und `Pentagon`. Entscheidend ist, dass wenn ein Programmierer nach dem Code für eine Klasse sucht, er genau sieht in welcher Datei dieser zu finden ist und nicht groß suchen muss.

Noch größere Projekte spalten man in mehrere *Module* indem man weitere Unterverzeichnisse anlegt. So wollen wir z.B. alle generischen Klassen für Punkte in das Verzeichnis `points` gruppieren und alle geometrischen Formen in das Verzeichnis `shapes`. So lassen sich größere Projekte gut strukturieren.

Die Summe aller Module formt dann ein *Package*. In diesem Fall das Package `geometry`, welches wir in verschiedenen Implementierungen wieder verwenden können.

- geometry
  - points
    - ImmutablePoint.py
    - Point.py
  - shapes
    - Line.py
    - Pentagon.py
    - Polygon.py
    - Tetragon.py
    - Triangle.py

Wenn eine KI Code für ein Projekt erzeugt, entsteht er in der Regel auf Modulebene: Ein Prompt wie „ein Modul, das Sensordaten verarbeitet“ liefert eine `.py`-Datei. Entscheidend ist, dass diese Datei ein klares öffentliches Interface hat, also die Funktionen und Klassen, die andere Module aufrufen dürfen. Interne Helfer werden mit führendem Unterstrich als privat markiert, und das Modul importiert nur, was es tatsächlich braucht.

Damit ist die Modulgrenze eine Architekturgrenze. Vor dem Übernehmen eines generierten Moduls stellen Sie drei Fragen: Was gehört in dieses Modul? Was wird zwischen den Modulen ausgetauscht? Und passt die Datei an die Stelle im Projekt, an der sie liegen soll?

Als Demo formulieren wir die Anforderung als Prompt: Das Modul `sensor_processing.py` soll eine Funktion `clean_readings` enthalten, die Werte außerhalb von 2σ entfernt. Die Typangabe

`list[float]` legt fest, dass die Funktion eine Liste aus Fließkommazahlen erwartet und wieder eine solche Liste zurückgibt. Die KI erzeugt daraufhin die folgende Datei.

```python
# sensor_processing.py  [KI-generiert]
import statistics

def clean_readings(values: list[float]) -> list[float]:
    if len(values) < 2:
        return values
    mean = statistics.mean(values)
    std  = statistics.stdev(values)
    return [v for v in values if abs(v - mean) <= 2 * std]

def _is_outlier(value: float, mean: float, std: float) -> bool:
    return abs(value - mean) > 2 * std
```

Beim Prüfen des generierten Moduls vergleichen wir zuerst das Interface mit dem Prompt: Funktionsname und Typangaben stimmen, `_is_outlier` ist privat markiert und es wird nur `statistics` importiert. Das Modul besteht den Interface-Check, hat aber zwei Schwächen. Erstens wird der Helfer `_is_outlier` von `clean_readings` nie aufgerufen, er ist toter Code. Zweitens liegt bei $n \leq 5$ Werten mit der Stichproben-Standardabweichung kein Wert außerhalb von 2σ (der größtmögliche Abstand beträgt $(n-1)/\sqrt{n}\,\sigma$, bei $n = 5$ also rund $1{,}79\sigma$). Der Aufruf `clean_readings([20.0, 20.1, 19.9, 20.0, 99.0])` liefert deshalb alle fünf Werte zurück. Beides fällt nur auf, wenn wir den Code lesen und mit eigenen Testwerten ausführen. Die Modulstruktur ist außerdem Teil der Review-Checkliste in `08b_CodeReview`.

`main()` - Der Startpunkt eines Programmes

Wenn Code über mehrere Dateien verteilt ist, braucht Python einen Hinweis, welcher Code ausgeführt werden soll. Hierfür definiert man die spezielle Einstiegsfunktion `main()`. Sie gibt es in fast allen Programmiersprachen und gibt immer den Startpunkt eines Programms an.

In Python hat sie entweder keine Argumente oder sie erhält diese dynamisch, wenn sie von anderen Prozessen aufgerufen wird, etwa über Kommandozeilenargumente.

```python
def main():
    print("This is the main function")
```

Allerdings möchte man vermeiden, dass die Funktion `main()` auch aufgerufen wird, wenn die Python-Datei zum Beispiel als Bibliothek eingebunden wird, wo man nur an den Funktionen interessiert ist. Deshalb nutzt man am Ende einer Datei mit einer `main()`-Funktion die folgende Verzweigung.

```python
if __name__ == "__main__":
    main()
```

```
This is the main function
```

Sie nutzt aus, dass der Wert der Standardvariable `__name__` in der Hauptdatei immer `'__main__'` lautet. In einer importierten Datei ist `__name__` dagegen der Modulname und eben nicht `'__main__'`.

Importieren von Modulen

Um zu vermeiden, dass ständig unnötiger Code geladen wird, lädt Python diesen Code nicht automatisch. Wollen wir also den Code in unseren Dateien, Modulen und Packages nutzen, so müssen wir Python vorher anweisen, diesen zu laden. Dieses *Importieren* weist man mit dem Befehl `import` an.

Ein einfacher Import ist der Import ganzer Module oder Packages. Dies geschieht, indem wir `import` und danach den Namen schreiben.

```
import geometry.points.ImmutablePoint
import geometry.shapes.Line

def main():
    point_1 = geometry.points.ImmutablePoint.ImmutablePoint(x=54.083336,
y=12.108811)
    point_2 = geometry.points.ImmutablePoint.ImmutablePoint(y=12.094167,
x=54.075211)
    linie_1 = geometry.shapes.Line.Line(start=point_1, end=point_2)
    print(f"Die Länge der Linie zwischen Punkt 1 und 2 ist:
{linie_1.length()}")

if __name__ == "__main__":
    main()
```

```
Die Länge der Linie zwischen Punkt 1 und 2 ist: 0.016747010509340444
```

Der Nachteil dieses Imports ist, dass wir für Klassen in Untermodulen den kompletten Pfad angeben müssen. In dem Beispiel oben also `geometry.points.ImmutablePoint.ImmutablePoint`.

Deshalb importiert man meist einzelne Module, indem man den Pfad eines Moduls angibt, wie zum Beispiel `geometry.points.ImmutablePoint`. Hierbei kann man den importierten Modulen auch neue Namen geben, wie `point` oder `line` im Beispiel unten.

```
import geometry.points.ImmutablePoint as point
import geometry.shapes.Line as line

def main():
    point_1 = point.ImmutablePoint(x=54.083336, y=12.108811)
    point_2 = point.ImmutablePoint(y=12.094167, x=54.075211)
```

```
    linie_1 = line.Line(start=point_1, end=point_2)
    print(f"Die Länge der Linie zwischen Punkt 1 und 2 ist:
{linie_1.length()}")

if __name__ == "__main__":
    main()
```

```
Die Länge der Linie zwischen Punkt 1 und 2 ist: 0.016747010509340444
```

Alternativ lassen sich auch Teile eines Moduls mit dem Platzhalter `*` und dem Befehl `from` importieren. Alle Elemente werden mit dem Platzhalter `*` importiert. Spezifische Elemente wie einzelne Klassen können auch direkt angegeben werden, wie im folgenden Beispiel `Line`.

Python sucht Module nicht „magisch“, sondern entlang seiner Importpfade. Diese stehen zur Laufzeit in `sys.path`. Dazu gehören typischerweise

- das aktuelle Projektverzeichnis
- die installierten Pakete der Python-Umgebung
- weitere Pfade, die z.B. durch Tools oder Umgebungsvariablen ergänzt wurden

Damit aus einem Verzeichnis wie `geometry/points/` ein importierbares Package wird, muss Python die Struktur als Package erkennen. In modernen Python-Versionen geht das oft schon über die Verzeichnisstruktur; häufig wird zusätzlich eine Datei `__init__.py` genutzt, um ein Package explizit zu markieren und zu konfigurieren.

Wichtig ist die Unterscheidung:

- *Verzeichnis:* nur eine Ordnerstruktur auf der Festplatte
- *Package:* ein Verzeichnis, das Python als importierbaren Namensraum behandelt
- *Modul:* eine konkrete Python-Datei wie `ImmutablePoint.py`

Wenn also `geometry` im Suchpfad liegt, kann Python ein Modul wie `geometry.points.ImmutablePoint` finden. Liegt das Projekt nicht im Suchpfad oder ist die Package-Struktur unklar, schlägt der Import fehl.

```
from geometry.points.ImmutablePoint import *
from geometry.shapes.Line import Line

def main():
    point_1 = ImmutablePoint(x=54.083336, y=12.108811)
    point_2 = ImmutablePoint(y=12.094167, x=54.075211)
    linie_1 = Line(start=point_1, end=point_2)
    print(f"Die Länge der Linie zwischen Punkt 1 und 2 ist:
{linie_1.length()}")

if __name__ == "__main__":
    main()
```

```
Die Länge der Linie zwischen Punkt 1 und 2 ist: 0.016747010509340444
```

Standard Pakete aus Python importieren

Python beinhaltet viele Standardpakete für typische Aufgaben. Für die Bau- und Umweltinformatik sind die folgenden am sinnvollsten:

package	description
collections	Mehr komplexe Datentypen zum zählen, sortieren
http	Funktionen des HTTP-Internetprotokolls wie Web-Server
json	Funktionen um Objekte als Text abzuspeichern
logging	Funktionen um Logs zu schreiben
math	Mathematische Funktionen
os	Funktionen um Dateien zu finden, laden und speichern
pickle	Funktionen um Objekte binär abzuspeichern
pprint	print-Funktionen um Objekte schöner (pretty) auszugeben
random	Funktionen zum Erzeugen von Zufallszahlen
re	Funktionen für Reguläre Ausdrücke um Text zu suchen
sys	Funktionen um Systeminformationen zu erhalten
time	Funktionen für Zeit und Datumsangaben
timeit	Funktionen um die Performance von Funktionen zu testen
traceback	Funktionen um den Stack aufzulisten
urllib	Funktionen, um URLs im Internet zu laden und zu verarbeiten

Von der Liste haben wir die Bibliotheken `math`, `time`, `timeit`, `traceback` und `logging` schon kennen gelernt und benutzt. Die anderen Pakete bieten allerdings weitere sinnvolle Funktionen.

Wollen wir zum Beispiel alle Dateien in einem Verzeichnis auflisten, so nutzen wir das Paket `os`.

```
import os

folder = "geometry/shapes/"
for count, filename in enumerate(os.listdir(folder)):
    if os.path.isfile(os.path.join(folder, filename)):
        path = os.path.join(folder, filename)
        print(path)
```

```
geometry/shapes/Tetragon.py
geometry/shapes/Pentagon.py
```

```
geometry/shapes/Line.py
geometry/shapes/Polygon.py
geometry/shapes/Triangle.py
```

Viele Webseiten bieten Programmierschnittstellen an, so genannte APIs, da diese APIs auch von den eigenen Webseiten genutzt werden, um Daten nachzuladen, die auf der Webseite angezeigt werden. Die APIs nutzen meist das JSON-Format um Daten auszutauschen. Dies ist ein Text-basiertes Dateiformat, das in Python sehr stark dem `dict`-Datentyp ähnelt, aber auch alle anderen primitiven und zusammengesetzten Datentypen von Python unterstützt.

Wir möchten zum Beispiel die Wetterdaten einer Wetterstation in Deutschland analysieren. Diese Daten bekommen wir beim Deutschen Wetterdienst. Diese Daten kann man sich auch von der API herunterladen. Hierfür braucht man die ID (Identifikationsnummer) einer Wetterstation, welche hier zu finden ist. Wir nehmen als Beispiel eine Station im Hansaviertel in Rostock mit der ID `12495`.

Dann kann man die Wetterdaten mit Python mit Hilfe des Paketes `urllib` von der API laden. Das JSON-Format können wir mit dem `json` Paket verarbeiten. Wir laden dazu zuerst vom Deutschen Wetterdienst ein binäres Datenpaket vom Datentype `bytes`. Dieses konvertieren wir mit der Funktion `loads` aus dem `json` Paket in ein Python `dict`. Um das schöner lesbar auszugeben, nutzen wir die Funktion `pprint` aus dem Paket `pprint` (pretty-print).

```
import urllib.request
import json
import pprint

stationID='12495' # Rostock-Hansaviertel
with urllib.request.urlopen(f'https://app-prod-ws.warnwetter.de/v30/stationOve
rviewExtended?stationIds={stationID}') as f:
    data=f.read() # Dies gibt uns ein binären Datentyp zurück
    wetter=json.loads(data) # Wir konvertieren den binären Datentyp in ein
dict
    pprint.pprint(wetter, indent=2, compact=True)
```

```
{ '12495': { 'days': [ { 'dayDate': '2026-10-18',
                         'icon': 4,
                         'icon1': None,
                         'icon2': None,
                         'moonPhase': 1,
                         'moonrise': 1792327403000,
                         'moonriseOnThisDay': 1792327403000,
                         'moonset': 1792355504000,
                         'moonsetOnThisDay': 1792355504000,
                         'precipitation': 0,
                         'stationId': None,
```

```
                        'sunrise': 1792299528000,
                        'sunriseOnThisDay': 1792299528000,
                        'sunset': 1792337504000,
                        'sunsetOnThisDay': 1792337504000,
                        'sunshine': 32767,
                        'temperatureMax': 95,
                        'temperatureMin': 49,
                        'windDirection': 3440,
                        'windGust': 259,
                        'windSpeed': 130},
                      { 'dayDate': '2026-10-19',
                        'icon': 7,
                        'icon1': None,
                        'icon2': None,
                        'moonPhase': 2,
                        'moonrise': 1792415146000,
                        'moonriseOnThisDay': 1792415146000,
                        'moonset': 1792446313000,
                        'moonsetOnThisDay': 1792446313000,
                        'precipitation': 20,
                        'stationId': None,
                        'sunrise': 1792386029000,
                        'sunriseOnThisDay': 1792386029000,
                        'sunset': 1792423780000,
                        'sunsetOnThisDay': 1792423780000,
                        'sunshine': 0,
                        'temperatureMax': 81,
                        'temperatureMin': 67,
                        'windDirection': 3600,
                        'windGust': 278,
                        'windSpeed': 111},
                      { 'dayDate': '2026-10-20',
                        'icon': 7,
                        'icon1': None,
                        'icon2': None,
                        'moonPhase': 2,
                        'moonrise': 1792502596000,
                        'moonriseOnThisDay': 1792502596000,
                        'moonset': 1792537185000,
                        'moonsetOnThisDay': 1792537185000,
                        'precipitation': 12,
                        'stationId': None,
                        'sunrise': 1792472531000,
                        'sunriseOnThisDay': 1792472531000,
                        'sunset': 1792510057000,
                        'sunsetOnThisDay': 1792510057000,
                        'sunshine': 0,
                        'temperatureMax': 116,
```

```
                          'temperatureMin': 76,
                          'windDirection': 1100,
                          'windGust': 296,
                          'windSpeed': 111},
                        { 'dayDate': '2026-10-21',
                          'icon': 3,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 2,
                          'moonrise': 1792589857000,
                          'moonriseOnThisDay': 1792589857000,
                          'moonset': 1792628110000,
                          'moonsetOnThisDay': None,
                          'precipitation': 0,
                          'stationId': None,
                          'sunrise': 1792559035000,
                          'sunriseOnThisDay': 1792559035000,
                          'sunset': 1792596335000,
                          'sunsetOnThisDay': 1792596335000,
                          'sunshine': 32767,
                          'temperatureMax': 180,
                          'temperatureMin': 92,
                          'windDirection': 1490,
                          'windGust': 463,
                          'windSpeed': 185},
                        { 'dayDate': '2026-10-22',
                          'icon': 7,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 2,
                          'moonrise': 1792677017000,
                          'moonriseOnThisDay': 1792677017000,
                          'moonset': 1792628110000,
                          'moonsetOnThisDay': 1792628110000,
                          'precipitation': 93,
                          'stationId': None,
                          'sunrise': 1792645538000,
                          'sunriseOnThisDay': 1792645538000,
                          'sunset': 1792682614000,
                          'sunsetOnThisDay': 1792682614000,
                          'sunshine': 32767,
                          'temperatureMax': 147,
                          'temperatureMin': 88,
                          'windDirection': 2230,
                          'windGust': 278,
                          'windSpeed': 93},
                        { 'dayDate': '2026-10-23',
                          'icon': 7,
```

```
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 3,
                          'moonrise': 1792764151000,
                          'moonriseOnThisDay': 1792764151000,
                          'moonset': 1792719104000,
                          'moonsetOnThisDay': 1792719104000,
                          'precipitation': 31,
                          'stationId': None,
                          'sunrise': 1792732042000,
                          'sunriseOnThisDay': 1792732042000,
                          'sunset': 1792768893000,
                          'sunsetOnThisDay': 1792768893000,
                          'sunshine': 32767,
                          'temperatureMax': 115,
                          'temperatureMin': 66,
                          'windDirection': 2860,
                          'windGust': 259,
                          'windSpeed': 111},
                        { 'dayDate': '2026-10-24',
                          'icon': 3,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 3,
                          'moonrise': 1792851339000,
                          'moonriseOnThisDay': 1792851339000,
                          'moonset': 1792810230000,
                          'moonsetOnThisDay': 1792810230000,
                          'precipitation': 0,
                          'stationId': None,
                          'sunrise': 1792818546000,
                          'sunriseOnThisDay': 1792818546000,
                          'sunset': 1792855174000,
                          'sunsetOnThisDay': 1792855174000,
                          'sunshine': 32767,
                          'temperatureMax': 117,
                          'temperatureMin': 58,
                          'windDirection': 2380,
                          'windGust': 222,
                          'windSpeed': 74},
                        { 'dayDate': '2026-10-25',
                          'icon': 3,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 3,
                          'moonrise': 1792938656000,
                          'moonriseOnThisDay': 1792938656000,
                          'moonset': 1792901574000,
```

```
                          'moonsetOnThisDay': 1792901574000,
                          'precipitation': 0,
                          'stationId': None,
                          'sunrise': 1792905050000,
                          'sunriseOnThisDay': 1792905050000,
                          'sunset': 1792941456000,
                          'sunsetOnThisDay': 1792941456000,
                          'sunshine': 32767,
                          'temperatureMax': 118,
                          'temperatureMin': 55,
                          'windDirection': 2150,
                          'windGust': 204,
                          'windSpeed': 93},
                         { 'dayDate': '2026-10-26',
                          'icon': 3,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 4,
                          'moonrise': 1793026215000,
                          'moonriseOnThisDay': 1793026215000,
                          'moonset': 1792993183000,
                          'moonsetOnThisDay': 1792993183000,
                          'precipitation': 0,
                          'stationId': None,
                          'sunrise': 1792991554000,
                          'sunriseOnThisDay': 1792991554000,
                          'sunset': 1793027739000,
                          'sunsetOnThisDay': 1793027739000,
                          'sunshine': 32767,
                          'temperatureMax': 113,
                          'temperatureMin': 62,
                          'windDirection': 2350,
                          'windGust': 204,
                          'windSpeed': 93},
                         { 'dayDate': '2026-10-27',
                          'icon': 3,
                          'icon1': None,
                          'icon2': None,
                          'moonPhase': 4,
                          'moonrise': 1793114210000,
                          'moonriseOnThisDay': 1793114210000,
                          'moonset': 1793084975000,
                          'moonsetOnThisDay': 1793084975000,
                          'precipitation': 2,
                          'stationId': None,
                          'sunrise': 1793078059000,
                          'sunriseOnThisDay': 1793078059000,
                          'sunset': 1793114023000,
```

```
                         'sunsetOnThisDay': 1793114023000,
                         'sunshine': 32767,
                         'temperatureMax': 88,
                         'temperatureMin': 46,
                         'windDirection': 3420,
                         'windGust': 259,
                         'windSpeed': 111}],
             'forecast1': { 'cloudCoverTotal': [],
                            'dewPoint2m': [ 54, 56, 57, 57, 58, 58, 60, 61,
61,
                                            62, 62, 62, 62, 63, 61, 62, 63,
64,
                                            66, 67, 67, 67, 67, 71, 72, 73,
74,
                                            74, 74, 78, 80, 82, 83, 82, 83,
82,
                                            87, 93, 93, 95, 99, 100, 99, 99,
                                            102, 100, 98, 97, 97, 95, 93, 91],
                            'humidity': [ 860, 890, 908, 914, 927, 914, 921,
                                          921, 927, 940, 946, 946, 946, 940,
                                          902, 890, 890, 890, 915, 966, 980,
                                          993, 1000, 993, 993, 993, 1000,
1000,
                                          986, 993, 993, 1000, 993, 987, 980,
                                          954, 935, 948, 917, 893, 893, 899,
                                          893, 911, 948, 967, 961, 954, 967,
                                          974, 973, 960],
                            'icon': [ 32767, 4, 32767, 32767, 32767, 32767,
                                      32767, 4, 32767, 32767, 32767, 32767,
                                      32767, 4, 32767, 32767, 32767, 32767,
                                      32767, 4, 32767, 4, 4, 4, 4, 4, 4, 4, 4,
                                      4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 7, 7,
7,
                                      7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 4, 4,
4,
                                      4, 7, 7, 4, 4, 4, 4, 7, 7, 7, 7, 4, 4,
4,
                                      4, 4],
                            'icon1h': [ 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
                                        4, 4, 4, 4, 4, 4, 7, 7, 7, 7, 7, 7, 7,
                                        7, 7, 7, 7, 7, 7, 7, 4, 4, 4, 4, 7, 7,
                                        4, 4, 4, 4, 7, 7, 7, 7, 4, 4, 4, 4,
4],
                            'isDay': [ False, False, False, False, False,
False,
                                       False, True, False, False, False,
False,
                                       False, True, False, False, False,
```

```
False,
                                    False, False, False, False, False,
False,
                                    False, False, False, False, False,
False,
                                    False, False, True, True, True, True,
                                    True, True, True, True, True, True,
                                    False, False, False, False, False,
False,
                                    False, False, False, False, False,
False,
                                    False, False, True, True, True, True,
                                    True, True, True, True, True, True,
                                    False, False, False, False, False,
False,
                                    False],
                          'precipitationProbablity': None,
                          'precipitationProbablityIndex': None,
                          'precipitationTotal': [ 32767, 32767, 32767,
32767,
                                                  32767, 32767, 32767,
32767,
                                                  32767, 32767, 32767,
32767,
                                                  32767, 32767, 32767,
32767,
                                                  32767, 32767, 32767,
32767,
                                                  32767, 0, 0, 0, 0, 0, 0,
0,
                                                  0, 0, 0, 0, 0, 0, 0, 0, 0,
                                                  0, 0, 0, 1, 1, 4, 4, 5, 1,
                                                  1, 1, 2, 2, 1, 1, 1, 1, 0,
                                                  0, 0, 0, 1, 1, 0, 0, 0, 0,
                                                  1, 1, 1, 1, 0, 0, 0, 0,
0],
                          'start': 1792274400000,
                          'stationId': '12495',
                          'sunshine': [ 32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
```

```
                                      32767, 32767],
                          'surfacePressure': [ 10270, 10270, 10272, 10271,
                                               10273, 10276, 10276, 10276,
                                               10277, 10280, 10284, 10287,
                                               10288, 10288, 10289, 10291,
                                               10289, 10292, 10291, 10293,
                                               10295, 10295, 10294, 10293,
                                               10289, 10287, 10285, 10281,
                                               10280, 10277, 10272, 10266,
                                               10262, 10261, 10263, 10261,
                                               10258, 10255, 10250, 10248,
                                               10246, 10241, 10239, 10236,
                                               10231, 10227, 10224, 10221,
                                               10214, 10212, 10208, 10205],
                          'temperature': [ 32767, 32767, 55, 32767, 32767,
                                           32767, 32767, 32767, 49, 32767,
                                           32767, 32767, 32767, 32767, 92,
                                           32767, 32767, 32767, 32767,
32767,
                                           77, 32767, 76, 73, 71, 70, 69,
71,
                                           72, 73, 72, 71, 70, 70, 70, 72,
76,
                                           79, 80, 81, 79, 72, 70, 68, 67,
72,
                                           73, 74, 74, 74, 76, 79, 81, 82,
84,
                                           84, 86, 89, 97, 101, 106, 112,
116,
                                           116, 116, 113, 110, 105, 104,
104,
                                           102, 99, 97, 97, 94, 95, 95, 94,
                                           92, 92, 94, 99, 112, 126, 140,
155,
                                           169, 177, 180, 173, 161, 152,
142,
                                           132, 129, 123, 121, 116, 110,
108,
                                           105, 104, 105, 104, 104, 107,
114,
                                           125, 134, 143, 146, 147, 146,
137,
                                           122, 115, 107, 103, 98, 96, 95,
88,
                                           86, 83, 81, 78, 76, 73, 72, 75,
80,
                                           89, 95, 102, 107, 112, 114, 115,
                                           107, 98, 89, 80, 74, 72, 69, 66,
```

```
                                          67, 64, 61, 60, 58, 59, 60, 66,
74,
                                          88, 100, 111, 117, 115, 114, 111,
                                          107, 100, 90, 84, 82, 77, 74, 72,
                                          69, 67, 64, 62, 58, 57, 55, 57,
67,
                                          81, 92, 103, 112, 116, 118, 115,
                                          104, 97, 89, 84, 83, 81, 78, 78,
                                          75, 72, 69, 68, 67, 68, 71, 71,
75,
                                          87, 97, 103, 108, 112, 113, 107,
                                          99, 91, 85, 77, 74, 71, 68, 65,
62,
                                          60, 56, 53, 51, 54, 55, 59, 62,
68,
                                          80, 79, 85, 87, 88, 85, 80, 73,
67,
                                          62, 55, 53, 51, 49, 46],
                           'temperatureStd': [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
                                             0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
                                             10, 11, 11, 12, 12, 11, 10, 9,
                                             9, 9, 9, 9, 9, 9, 10, 11, 9,
9,
                                             10, 10, 10, 10, 9, 9, 11, 11,
                                             11, 11, 11, 12, 12, 12, 11,
12,
                                             11, 10, 10, 11, 13, 12, 12,
14,
                                             14, 14, 14, 13, 12, 10, 9, 10,
                                             11, 12, 12, 13, 13, 13, 12,
12,
                                             11, 12, 12, 14, 15, 14, 15,
15,
                                             15, 15, 13, 12, 12, 13, 14,
14,
                                             14, 12, 13, 14, 14, 15, 15,
15,
                                             15, 15, 15, 16, 16, 17, 19,
19,
                                             20, 20, 18, 15, 15, 15, 15,
14,
                                             14, 13, 12, 12, 14, 14, 14,
14,
                                             15, 15, 15, 15, 14, 16, 15,
13,
                                             12, 14, 13, 13, 13, 14, 15,
```

```
15,
                                            16, 17, 16, 15, 15, 15, 17,
17,
                                            17, 17, 16, 15, 16, 16, 17,
18,
                                            17, 17, 14, 14, 14, 14, 15,
15,
                                            15, 16, 16, 16, 16, 16, 16,
16,
                                            17, 16, 15, 14, 14, 17, 17,
17,
                                            18, 18, 16, 16, 16, 17, 17,
17,
                                            18, 18, 18, 18, 18, 17, 18,
17,
                                            17, 17, 16, 15, 15, 18, 20,
20,
                                            20, 20, 19, 18, 17, 18, 17,
17,
                                            17, 17, 19, 20, 20, 19, 19,
19,
                                            19, 19, 17, 16, 18, 19, 20,
19,
                                            18, 18, 17, 17, 16, 16, 17,
18,
                                            17, 17, 18],
                          'timeStep': 3600000,
                          'windDirection': None,
                          'windGust': None,
                          'windSpeed': None},
             'forecast2': { 'cloudCoverTotal': [],
                          'dewPoint2m': [ 82, 79, 88, 103, 103, 102, 92, 88,
                                          87, 84, 96, 101, 101, 97, 87, 83,
                                          78, 70, 68, 75, 73, 71, 63, 59,
56,
                                          51, 60, 68, 68, 65, 59, 53, 47,
45,
                                          56, 62, 64, 62, 56, 55, 53, 55,
66,
                                          72, 59, 56, 53, 47, 44, 44, 49,
42,
                                          44, 40, 38, 36],
                          'humidity': [ 916, 916, 852, 712, 607, 721, 782,
                                        829, 886, 874, 887, 759, 744, 887,
                                        929, 967, 980, 980, 921, 833, 759,
                                        833, 927, 953, 966, 946, 908, 748,
                                        733, 789, 854, 877, 901, 933, 842,
                                        713, 709, 832, 842, 871, 902, 896,
```

```
                                    867, 784, 722, 820, 883, 901, 939,
                                    926, 877, 744, 754, 829, 900, 932],
                          'icon': [ 4, 4, 4, 3, 2, 2, 2, 2, 2, 2, 3, 7, 7,
8,
                                    8, 7, 7, 7, 4, 4, 4, 3, 7, 7, 2, 2, 3,
2,
                                    3, 3, 2, 3, 4, 4, 4, 3, 3, 3, 2, 2, 2,
3,
                                    3, 3, 3, 3, 3, 3, 4, 4, 3, 3, 3, 3, 7,
                                    7],
                          'icon1h': [ 4, 4, 4, 3, 2, 2, 2, 2, 2, 2, 3, 7, 7,
                                      8, 8, 7, 7, 7, 4, 4, 4, 3, 7, 7, 2, 2,
                                      3, 2, 3, 3, 2, 3, 4, 4, 4, 3, 3, 3, 2,
                                      2, 2, 3, 3, 3, 3, 3, 3, 3, 4, 4, 3, 3,
                                      3, 3, 7, 7],
                          'isDay': [ False, False, True, True, True, True,
                                     False, False, False, False, True, True,
                                     True, True, False, False, False, False,
                                     True, True, True, True, False, False,
                                     False, False, True, True, True, True,
                                     False, False, False, False, True, True,
                                     True, False, False, False, False,
False,
                                     True, True, True, False, False, False,
                                     False, False, True, True, True, False,
                                     False, False],
                          'precipitationProbablity': None,
                          'precipitationProbablityIndex': None,
                          'precipitationTotal': [ 0, 0, 0, 0, 0, 0, 0, 0, 0,
                                                  0, 0, 8, 14, 26, 27, 18,
16,
                                                  6, 0, 0, 0, 0, 3, 6, 0, 0,
                                                  0, 0, 0, 0, 0, 0, 0, 0, 0,
                                                  0, 0, 0, 0, 0, 0, 0, 0, 0,
                                                  0, 0, 0, 0, 0, 0, 0, 0, 0,
                                                  0, 1, 1],
                          'start': 1792533600000,
                          'stationId': '12495',
                          'sunshine': [ 32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
                                        32767, 32767, 32767, 32767, 32767,
```

```
                                    32767, 32767, 32767, 32767, 32767,
                                    32767],
                    'surfacePressure': [ 10195, 10182, 10175, 10168,
                                         10159, 10158, 10154, 10151,
                                         10149, 10154, 10160, 10166,
                                         10168, 10178, 10186, 10197,
                                         10200, 10207, 10212, 10202,
                                         10187, 10184, 10184, 10180,
                                         10174, 10169, 10178, 10179,
                                         10170, 10174, 10182, 10178,
                                         10176, 10177, 10178, 10170,
                                         10158, 10158, 10157, 10150,
                                         10139, 10126, 10147, 10143,
                                         10147, 10150, 10156, 10142,
                                         10144, 10152, 10149, 10143,
                                         10126, 10128, 10138, 10139],
                    'temperature': [],
                    'temperatureStd': [],
                    'timeStep': 10800000,
                    'windDirection': None,
                    'windGust': None,
                    'windSpeed': None},
          'forecastStart': None,
          'threeHourSummaries': None,
          'warnings': []}}
```

Externe Pakete installieren und importieren

Die Stärke von Python ist die riesige Auswahl an vorhandenen Paketen. Für die meisten Anwendungszwecke gibt es passende Python-Packages. Die wichtigste öffentliche Quelle dafür ist PyPI, der Python Package Index.

Die Standardbibliothek gehört bereits zu Python und ist sofort verfügbar. Pakete aus PyPI werden zusätzlich in die aktive Python-Umgebung installiert. Genau diese Trennung sollte man im Projekt immer im Blick behalten: „mit Python mitgeliefert“ ist nicht dasselbe wie „erst nachinstalliert“.

Die Installation neuer Pakete erfolgt im Terminal, nicht innerhalb eines Python-Skripts. Dafür nutzt man typischerweise einen Befehl wie `pip install <paketname>`.

Zum Beispiel wollen wir die eben geladenen Wetterdaten anzeigen. Das Beispiel soll vor allem zeigen, wie externe Pakete zusammenspielen. Hierfür nutzen wir:

- zuerst das Paket `pandas` zum Erzeugen einer Tabelle aus den Wetterdaten
- dann nutzen wir das Paket `plotly` zum Zeichnen eines Diagramms
- zuletzt erzeugen wir mit `dash` optional einen kleinen Webserver, der das Diagramm anzeigt

Alles drei installieren wir mit `pip`. Wir können das in einem Aufruf machen. Wir nutzen den `--quiet` Switch, um die Ausgabe zu reduzieren.

```
{bash}
#| echo: true
pip install pandas plotly dash --quiet
```

Jetzt laden wir die installierten Pakete, wobei man üblicherweise `pandas` die Abkürzung `pd` zuweist und Plotly Express die Abkürzung `px`.

```
import pandas as pd
import plotly.express as px
```

Nun wandeln wir die tageweise Wettervorhersage `days` der Daten von der Wetterstation mit der `stationID` zuerst in eine Tabelle um, da diese von Plotly Express verarbeitet werden kann. Tabellen heißen in Pandas DataFrames. Wir erzeugen also aus der Wettervorhersage eine neue Objektinstanz vom Typ `DataFrame` via

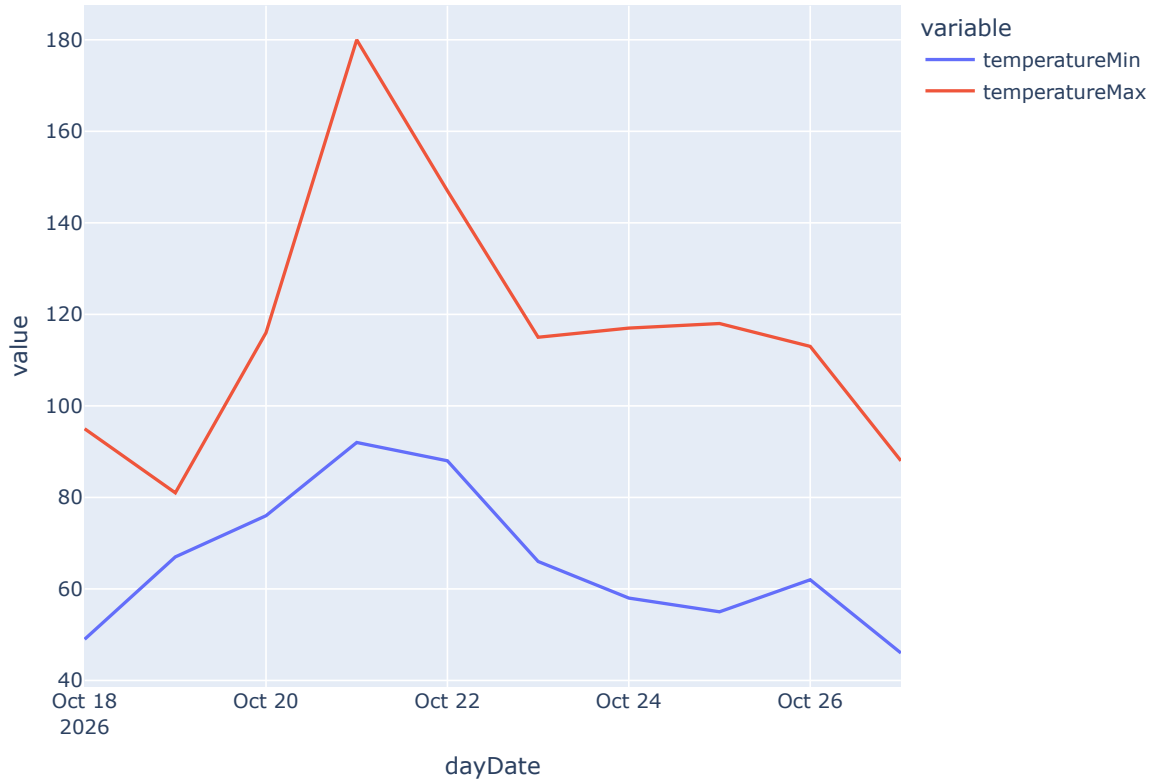
```
df = pd.DataFrame(wetter[stationID]['days'])
df
```

	sta-tio-nId	day-Da-te	tem-pe-ra-tu-re-Min	tem-pe-ra-tu-re-Max	wind-pi-ta-tion	windSpeed	windGust	winds-Di-rec-tion	suns-hi-ne	sun-ri-se	moon-ri-se	moon-set	moon-se-tOn-This-Day	moon-sun-se-tOn-This-Day	sun-ri-se-tOn-This-Day	moon-se-Pha-se	icon1	icon2
0	None	2026-01-08	0	130	259	34403276	7792299	5	1	7	4	0	0	0	0	0.94	None	None
1	None	2026-01-09	1	20	111	278	36000	7792386	0	7	5	1	1	1	1	0.97	None	None
2	None	2026-01-10	16	12	111	296	11000	7792472	5	7	6	2	2	2	2	0.00	None	None
3	None	2026-01-11	80	0	185	463	14903276	7792559	0	7	6	3	3	3	3	0.03	None	None
4	None	2026-01-12	47	93	93	278	22303276	7792645	5	7	7	4	4	4	4	0.06	None	None
5	None	2026-01-13	15	31	111	259	28603276	7792732	0	7	8	5	5	5	5	0.09	None	None
6	None	2026-01-14	17	0	74	222	23803276	7792818	5	7	9	6	6	6	6	0.12	None	None

	stationId	dayDate	temperatureMin	temperatureMax	precipitation	windSpeed	windGust	windDirection	sunsetOnThisDay	sunriseOnThisDay	moonriseOnThisDay	moonsetOnThisDay	moonsetOnDayAfterThisDay	sunriseOnDayAfterThisDay	sunsetOnDayAfterThisDay	moonriseOnDayAfterThisDay	moonPhase	icon	icon1	icon2
7	None	2026-10-25	11	18	0	93	204	215	1792	1790	1790	1790	1790	1790	1790	1790	0.5	None	None	None
8	None	2026-10-26	12	21	0	93	204	235	1792	1791	1790	1790	1790	1790	1790	1790	0.5	None	None	None
9	None	2026-10-27	8	2	2	111	259	342	1793	1792	1790	1790	1790	1790	1790	1790	0.5	None	None	None

Nun plotten wir die Daten `df` als Liniendiagramm mit Hilfe von Plotly, wobei wir als `x`-Achse das Datum wählen (`dayDate`) und als `y`-Achse die minimale Temperatur `temperatureMin` und maximale Temperatur `temperatureMax`.

```
fig=px.line(df, x="dayDate", y=["temperatureMin", "temperatureMax"])
fig.show(renderer="svg")
```



Optional können wir dieses Diagramm anschließend in einer Webseite auf einem Webserver anzeigen. Hier nutzen wir das Paket `dash`, das erlaubt, eine Webseite mit Python-Befehlen zu erzeugen und darin interaktive Diagramme von Plotly anzuzeigen.

```
import dash

app = dash.Dash()
```

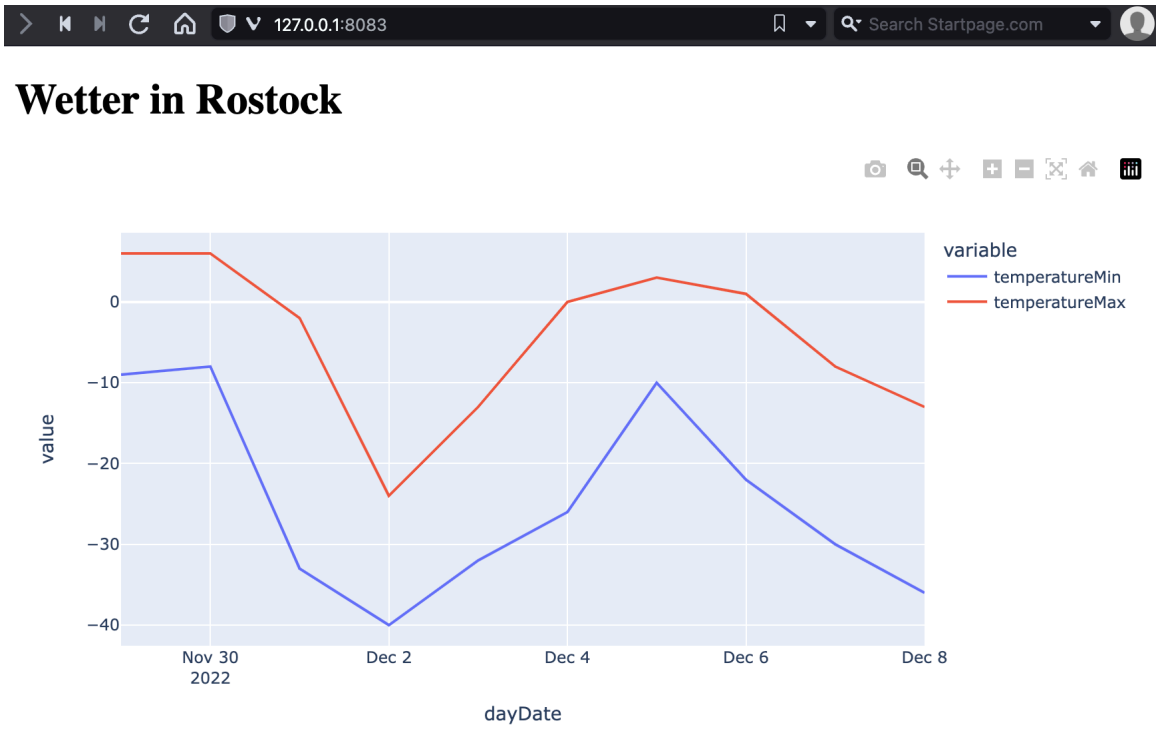
Dann erzeugen wir eine Webseite mit einer Überschrift (`H1`), welche den Plot als `Graph` enthält.

```
app.layout = dash.html.Div(children = [
    dash.html.H1(children='Wetter in Rostock'),
    dash.dcc.Graph(id="wetter", figure=fig)
])
```

Und starten den Webserver.

```
app.run_server()
```

Dies startet einen Webserver, auf den wir in einem Browser unter `http://127.0.0.1:8083/` zugreifen können. Er zeigt eine Webseite mit einem interaktiven Diagramm der Wetterdaten.



Ein Package kann außerdem festlegen, welche Namen beim Import nach außen sichtbar sein sollen. Dazu kann in `__init__.py` oder in einem Modul die Variable `__all__` definiert werden.

Sie ist besonders dann hilfreich, wenn ein Package ein bewusst kleines öffentliches Interface anbieten soll.

```
__all__ = ["Line", "Triangle", "Polygon"]
```

Die Konvention mit führendem Unterstrich (`_helper`) sagt also: „intern gedacht“. `__all__` macht dieses öffentliche Interface zusätzlich explizit.

Python-Ökosystem: Standardbibliothek und PyPI

Ein großer Teil der Stärke von Python liegt im Zusammenspiel aus Standardbibliothek und externer Paketwelt. Die Standardbibliothek deckt viele grundlegende Aufgaben ab, etwa Dateizugriffe, JSON-Verarbeitung oder mathematische Funktionen. PyPI erweitert dieses Fundament um spezialisierte Bibliotheken für Datenanalyse, Visualisierung, Webentwicklung, Geometrie, Machine Learning und vieles mehr.

Für die Projektarbeit ist diese Unterscheidung wichtig:

- *Standardbibliothek:* sofort nach der Python-Installation nutzbar
- *PyPI-Pakete:* müssen pro Umgebung installiert und versioniert werden

Gerade in Teamprojekten und bei KI-generiertem Code sollte man deshalb immer prüfen, ob ein Import schon durch Python verfügbar ist oder ob die Abhängigkeit erst dokumentiert und installiert werden muss.

Bibliographie